

Trabalho Prático 3 - Análise de Livros: Centralidade em uma Rede Complexa

Nicolas Rodrigues, Pedro Finochio, Jeann Victor, Thallyson Luis e Thiago Martins

¹Departamento de Ciências da Computação – Universidade Federal de Alfenas
Avenida Jovino Fernandes de Sales 2600 – CEP 37133840 - Alfenas – MG - Brasil

nicolas.oliveira@sou.unifal-mg.edu.br pedro.finochio@sou.unifal-mg.edu.br

jeann.batista@sou.unifal-mg.edu.br thallyson.luis@sou.unifal-mg.edu.br

thiagomartins.silva@sou.unifal-mg.edu.br

Resumo. Neste trabalho, foi implementado um algoritmo capaz de ler o livro *A Storm of Swords*, de George R. R. Martin, e **construir uma rede complexa de relacionamentos entre os personagens da obra**, resultando em um grafo que representa a proximidade entre eles com base na coocorrência de seus nomes ao longo do texto. Para viabilizar a aplicação de algoritmos de caminhos mínimos, foi desenvolvido um método de inversão proporcional dos pesos das arestas, convertendo relações mais frequentes em custos menores. Além disso, **foi calculada a centralidade betweeness dos vértices**, permitindo identificar e ordenar os personagens mais centrais na estrutura narrativa.

1. Introdução

Este trabalho propõe a análise do Problema da Centralidade em Grafos a partir do livro **A Storm of Swords**, da série *Game of Thrones*, onde os vértices representam personagens e as arestas indicam relações entre eles, baseadas na proximidade de seus nomes no texto.

A análise de **centralidade em grafos** busca identificar **quais vértices exercem maior influência ou importância dentro da rede**, medindo, por exemplo, a frequência com que um vértice aparece em caminhos críticos entre outros vértices. **Essa métrica é fundamental para compreender a estrutura e a dinâmica do sistema representado pelo grafo.**

O objetivo central é **identificar os personagens mais relevantes por meio da métrica de centralidade betweeness**, além de **calcular o circuit rank da rede para compreender sua estrutura cíclica**. Para isso, foram desenvolvidos algoritmos específicos em C, abordando leitura do texto, construção da rede, cálculo de centralidade e análise estrutural do grafo.

2. Algoritmos

Foram implementados os principais algoritmos para construir e analisar a rede complexa de personagens:

Comparação de Strings Ignorando Pontuação

Permite comparar nomes de personagens mesmo com pontuações diferentes, garantindo identificação correta.

```
1 bool compSemPontu(char *str1, const char *str2) {
2     while (*str1 != '\0' && *str2 != '\0') {
3         if (*str1 < 0 || ePontuacao(*str1)) {
4             str1++;
5             continue;
6         }
7         if (*str1 != *str2) {
8             return false;
9         }
10        str1++;
11        str2++;
12    }
13    return (*str1 != '\0' || *str2 == '\0');
14 }
```

Listing 1. Função que compara strings ignorando pontuações

Vetor Dinâmico para Armazenamento de Ocorrências

Armazena as posições dos personagens no texto com redimensionamento automático.

```
1 typedef struct vetor_{
2     int tam_alocado, tam;
3     posicoes *elm;
4 } vetor;
5
6 vetor *iniciaVet(int tam_inicial);
7 void insereVet(vetor *vet, posicoes valor);
```

Listing 2. Estrutura e funções para vetor dinâmico

Construção da Matriz de Relações

Matriz de adjacência onde o peso representa a frequência de proximidade entre personagens no texto (até 100 palavras).

Inversão Proporcional dos Pesos das Arestas

Inverte os pesos para que maior frequência resulte em menor custo para algoritmos de caminho mínimo:

$$x' = 1/x$$

Algoritmo de Dijkstra com Heap Mínima

Implementação eficiente do Dijkstra usando heap para seleção rápida do próximo vértice.

```

1 void siftUp(int heap[], int pos[], int i, int dist[]);
2 void siftDown(int heap[], int pos[], int i, int tamHeap, int
  dist[]);
3 void insereHeap(int heap[], int pos[], int v, int *tamHeap, int
  dist[]);
4 int removeMin(int heap[], int pos[], int *tamHeap, int dist[]);

```

Listing 3. Operações de heap para Dijkstra

Cálculo da Centralidade Betweenness

Baseado no algoritmo de Brandes, calcula a centralidade para cada vértice usando múltiplas execuções de Dijkstra. A seguir, temos o pseudocódigo do algoritmo.

Algorithm 1 Brandes(Graph)	
1: for cada nó v em V do	22: $P[w] \leftarrow [v]$
2: $C_B[v] \leftarrow 0$	23: $PQ.enqueue(w, dist[w])$
3: end for	24: else if $dist[w] = dist[v] + weight(v, w)$ then
4: for cada nó s em V do	25: $\sigma[w] \leftarrow \sigma[w] + \sigma[v]$
5: $S \leftarrow$ pilha vazia	26: $P[w].append(v)$
6: $PQ \leftarrow$ fila de prioridade vazia	27: end if
7: for cada nó t em V do	28: end for
8: $P[t] \leftarrow []$	29: end while
9: $\sigma[t] \leftarrow 0$	30: $\delta \leftarrow \{v : 0 \text{ para todo } v \in V\}$
10: $dist[t] \leftarrow \infty$	31: while S não está vazia do
11: end for	32: $w \leftarrow S.pop()$
12: $\sigma[s] \leftarrow 1$	33: for cada v em $P[w]$ do
13: $dist[s] \leftarrow 0$	34: $\delta[v] \leftarrow \delta[v] + \left(\frac{\sigma[v]}{\sigma[w]}\right) \cdot (1 + \delta[w])$
14: $PQ.enqueue(s, 0)$	35: end for
15: while PQ não está vazia do	36: if $w \neq s$ then
16: $v \leftarrow PQ.extract_min()$	37: $C_B[w] \leftarrow C_B[w] + \delta[w]$
17: $S.push(v)$	38: end if
18: for cada vizinho w de v do	39: end while
19: if $dist[w] > dist[v] + weight(v, w)$ then	40: end for
20: $dist[w] \leftarrow dist[v] + weight(v, w)$	41: for cada nó v em V do
21: $\sigma[w] \leftarrow \sigma[v]$	42: $C_B[v] \leftarrow C_B[v] \div 2$
	43: end for
	44: return C_B

Figura 1. Pseudocódigo do algoritmo de Brandes para cálculo da centralidade entre vértices em grafos.

O algoritmo de Brandes possui complexidade de tempo:

$$\mathcal{O}((n + m) \cdot n \log n)$$

onde $n = |V|$ e $m = |E|$.

Cálculo do Circuit Rank

Calculado pela fórmula:

$$r = e - v + c$$

onde e é o número de arestas, v o número de vértices e c o número de componentes conexos.

Para obter o número de componentes conexos foi utilizado um algoritmo de busca em profundidade (DFS), cuja complexidade é:

$$\mathcal{O}(n + m)$$

onde n é o número de vértices e m o número de arestas.

Algorithm 1 DFS_VISIT(<i>grafo</i> , <i>n</i> , <i>visitado</i> , <i>u</i>)	
1:	<i>visitado</i> [<i>u</i>] ← verdadeiro
2:	for <i>v</i> ← 0 até <i>n</i> − 1 do
3:	if ¬ <i>visitado</i> [<i>v</i>] e <i>grafo</i> [<i>u</i>][<i>v</i>] ≠ 0 then
4:	DFS_VISIT(<i>grafo</i> , <i>n</i> , <i>visitado</i> , <i>v</i>)
5:	end if
6:	end for

Algorithm 2 NUM_COMPONENTES_CONEXOS(<i>grafo</i> , <i>n</i>)	
1:	criar vetor <i>visitado</i> [0 . . . <i>n</i> − 1] ← falso
2:	<i>count</i> ← 0
3:	for <i>u</i> ← 0 até <i>n</i> − 1 do
4:	if ¬ <i>visitado</i> [<i>u</i>] then
5:	DFS_VISIT(<i>grafo</i> , <i>n</i> , <i>visitado</i> , <i>u</i>)
6:	<i>count</i> ← <i>count</i> + 1
7:	end if
8:	end for
9:	return <i>count</i>

Figura 2. Pseudocódigo do algoritmo implementado para cálculo do Circuit Rank.

3. Resultados

Abaixo, apresentamos a matriz de relações entre os personagens com base na coocorrência de nomes:

Tabela 1. Matriz de proximidade entre personagens

	Arya	Bran	Brienne	Catelyn	Cersei	Jaime	Sam	Sansa	Tyrion	Varys
Arya	0	25	12	34	18	28	2	71	16	8
Bran	25	0	2	49	5	10	86	46	17	0
Brienne	12	2	0	42	44	418	0	17	31	2
Catelyn	34	49	42	0	16	83	4	47	14	5
Cersei	18	5	44	16	0	208	8	114	261	30
Jaime	28	10	418	83	208	0	2	49	176	30
Sam	2	86	0	4	8	2	0	0	5	4
Sansa	71	46	17	47	114	49	0	0	303	31
Tyrion	16	17	31	14	261	176	5	303	0	127
Varys	8	0	2	5	30	30	4	31	127	0

Centralidade Betweenness

- Arya: 0.000000
- Bran: 0.222222
- Brienne: 0.000000
- Catelyn: 0.000000
- Cersei: 0.000000
- Jaime: 0.333333
- Sam: 0.000000
- Sansa: 0.472222
- Tyrion: 0.583333
- Varys: 0.000000

Circuit Rank e Métricas da Rede

- Número de componentes conexos: **1**
- Número de arestas: **42**
- Circuit Rank: **33**
- Nomes encontrados no texto: **3728**
- Tempo de execução: **0.048450 segundos**

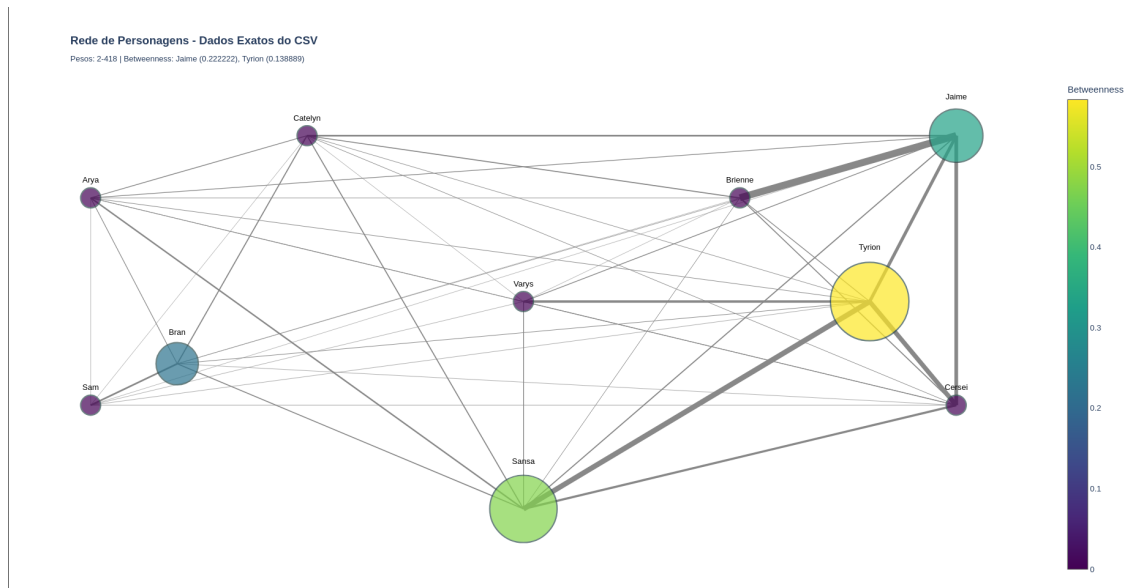


Figura 3. Representação do Grafo

O circuito rank elevado (33) indica uma rede altamente cíclica, o que é esperado em um enredo complexo como o de Game of Thrones, onde personagens frequentemente interagem em múltiplos contextos. A existência de um único componente conexo confirma que todos os personagens estão, direta ou indiretamente, interligados.

4. Descrição do Makefile e Execução

O projeto foi desenvolvido em linguagem C, com múltiplos arquivos fonte e organização modular. O Makefile automatiza os processos de compilação, linkagem e limpeza do projeto, além de criar diretórios de objetos e dependências.

Makefile do Projeto

```
1 # Nome do projeto
2 PROJ_NAME = ProjetoGameOfTrones.bin
3
4 # Diretório para arquivos .o e .d
5 OBJ_DIR = obj
6
7 # Arquivos fonte e objetos
8 C_SOURCE = $(wildcard *.c)
9 OBJ = $(patsubst %.c, $(OBJ_DIR)/%.o, $(C_SOURCE))
10
```

```

11 # Compilador e flags
12 CC = gcc
13 CC_FLAGS = -O2 -std=c99 -c -Wall -Wextra -MMD -MP -lm
14
15 # Compila o e linkagem
16 all: $(PROJ_NAME)
17
18 $(PROJ_NAME): $(OBJ)
19     $(CC) $^ -o $@ -lm
20
21 $(OBJ_DIR)/%.o: %.c | $(OBJ_DIR)
22     $(CC) $(CC_FLAGS) -o $@ $<
23
24 # Cria o do diretório de objetos
25 $(OBJ_DIR):
26     mkdir -p $(OBJ_DIR)
27
28 # Limpeza dos arquivos gerados
29 clean:
30     rm -rf $(OBJ_DIR) *~ $(PROJ_NAME)
31
32 # Inclusão automática de dependências
33 -include $(OBJ:.o=.d)

```

Listing 4. Makefile do projeto

Comandos Principais

- **Compilar o projeto:**
make all
- **Executar o programa:**
./ProjetoGameOfTrones.bin input.txt
- **Limpar os arquivos gerados:**
make clean

5. Referências

Referências

- [1] Brandes, U. (2001). A faster algorithm for betweenness centrality. *Journal of Mathematical Sociology*, 25(2):163–177.
- [2] Newman, M. E. J. (2010). *Networks: An Introduction*. Oxford University Press.
- [3] Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
- [4] Freeman, L. C. (1977). A set of measures of centrality based on betweenness. *Sociometry*, 40(1):35–41.